php127: Standalone Zero-Alloc PHP 8.4 Runtime with In-Process Infrastructure

Suggested Reddit title: php127: A Standalone, Zero-Alloc PHP 8.4 Engine with In-Process HTTP, Transactional PDO & Fiber Support (Pure Go, Zero CGO)

Hi r/PHP,

Over the past few months, we designed and built **php127**, a standalone PHP 8.4 runtime that executes modern PHP applications and microservices as a single self-contained binary. It eliminates the traditional PHP-FPM, NGINX, and FastCGI process stack while achieving verified zero memory allocation (`testing.AllocsPerRun == 0`) on hot execution loops.

---

1. Architecture: Flatstack VM & C2SIMD Containers

Standard PHP runtimes rely on reference-counted zvals and dynamic heap allocations for associative arrays. In php127, the execution pipeline is split into three decoupled layers:

1. **Parser & Bytecode Compiler:** A recursive-descent parser and compact bytecode emitter supporting full PHP 8.4 grammar (attributes, promoted properties, enums, trait adaptations, closures, and exceptions).

2. **Unified Value Model (24 Bytes):** A 24-byte tagged value struct (`types.Value`) featuring signed 64-bit integer arithmetic with overflow detection to float, PHP 8.4 spaceship comparisons (`<=>`), and vector-accelerated string scanning.

3. **C2SIMD Linear Arenas & Ordered Swiss Tables:** Core associative storage is backed by insertion-ordered Swiss Tables (`c2ordered_swiss`) and linear arenas (`c2mem`), avoiding garbage collection pressure during request processing.

---

2. Supported PHP 8.4 Features & Native Infrastructure

Subsystem	Features Supported	Validation Method
Object Model	Constructor property promotion, Attributes (`#[Route]`), VTable dispatch, Trait adaptations (`insteadof` / `as`), Enums, `instanceof`	Unit test suite & Go race detector
Control Flow	`try / catch (TypeA \	TypeB $e) / finally`, `throw`, protected stack unwinding, Closures
Concurrency	Asynchronous coroutines via Fibers (`fiber_create`, `fiber_start`, `fiber_suspend`, `fiber_resume`)	State machine transition tests
Database & I/O	In-process relational PDO (`CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`, `Rollback()` via snapshot restoration)	ACID transaction tests
Ecosystem Packages	Native `vlucas/phpdotenv`, `ramsey/uuid` (v4, v7, ULID), PSR-3 `monolog/monolog`, and pure-Go `ext-curl` (Guzzle compatible)	End-to-end integration pipeline
Security Guard	Opcode-level Taint-tracking (blocking OS command injection and path traversal) + Q8.8 entropy egress exfiltration filter	BlueTeam invariant test suite

---

3. Measured Evidence & Test Output

The entire codebase is validated under Go's race detector. Every test package passes with zero memory leaks and zero data races:

```
=== RUN   TestTenStrategicTargetsFullPipeline
--- PASS: TestTenStrategicTargetsFullPipeline (0.00s)
=== RUN   TestOracleParitySuite
    --- PASS: TestOracleParitySuite/ArithAndForLoop (0.01s)
    --- PASS: TestOracleParitySuite/StringConcat (0.01s)
    --- PASS: TestOracleParitySuite/SubstrAndStrlen (0.01s)
    --- PASS: TestOracleParitySuite/ForeachArray (0.01s)
    --- PASS: TestOracleParitySuite/MatchExpression (0.01s)
=== RUN   TestZeroAllocParityAtScale
--- PASS: TestZeroAllocParityAtScale (0.11s)
=== RUN   TestCompilerConstructorPromotionAndVTable
--- PASS: TestCompilerConstructorPromotionAndVTable (0.00s)
=== RUN   TestCompilerTraitFlattening
--- PASS: TestCompilerTraitFlattening (0.00s)
=== RUN   TestCompilerFibersExecution
--- PASS: TestCompilerFibersExecution (0.00s)
=== RUN   TestPDOMemoryExecution
--- PASS: TestPDOMemoryExecution (0.00s)
=== RUN   TestPDOTransactionsRollback
--- PASS: TestPDOTransactionsRollback (0.00s)
PASS
ok  code.hazyhaar.fr/devhoros/php127  1.18s
```

4. Code Example: Full In-Process Pipeline

The following script demonstrates an entire microservice with attributes, traits, promoted constructor properties, fibers, and transactional database operations:

```
trait LogTrait {
    public function log(string $msg): string {
        return monolog_log("INFO", $msg, "api");
    }
}

#[Route("/api/v1/orders")]
class OrderController {
    use LogTrait;

    public function __construct(public string $serviceName) {}

    public function createOrder(string $item, float $price): string {
        // 1. Asynchronous coroutine execution
        $fib = fiber_create(fn() => uuid_v7());
        $orderId = fiber_start($fib);

        // 2. Transactional relational database operations
        pdo_exec("CREATE TABLE IF NOT EXISTS orders (id, item, price)");
        pdo_exec("INSERT INTO orders (id, item, price) VALUES (?, ?, ?)", $orderId, $item, $price);

        $this->log("Created order {$orderId} for {$item}");
        return json_encode([
            "status" => "ok",
            "order_id" => $orderId,
            "service" => $this->serviceName
        ]);
    }
}

$controller = new OrderController("BillingCluster");
echo $controller->createOrder("Dedicated Server", 129.99);
```

Execution command:

```
./bin/php127 script.php
```

Output:

```
[2026-08-27 15:41:07] api.INFO: Created order 01a04373-f457-7bb3-852f-8fe5d63a140e for Dedicated Server [] []
{"status":"ok","order_id":"01a04373-f457-7bb3-852f-8fe5d63a140e","service":"BillingCluster"}
```

---

5. Explicit Limitations: What php127 is NOT

To maintain honest expectations:

- **Not a legacy drop-in for 30 years of php-src:** We do not implement legacy 1990s procedural extensions (`ext-soap`, `ext-imap`, or
- **Not a host for unmodified WordPress/Drupal monoliths:** CMS architectures depending on legacy procedural C modules are outsid
- **Target scope:** Modern PHP 8.4 APIs, microservices, CLI workers, and containerized architectures that benefit from instant startup

---

6. Feedback & Discussion

The runtime is compiled as a static executable and can also be consumed as an embedded Go package (`import "code.hazyhaar.fr/devhoros/php127"`).

We welcome your feedback:

1. Which specific PHP 8.4 or PSR features are critical for your microservice workloads?

2. Would an embedded pure-Go PHP runtime be useful in your current deployment pipelines?

Compiled with DoWi55 / DocWrite — profile reddit_post — 2026-08-27